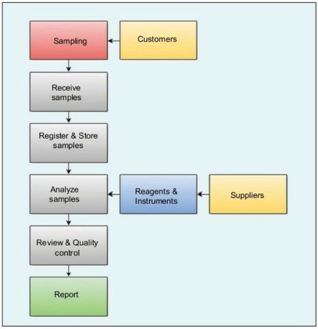

Figure 1: Workflow

- MariaDB, as database engine, available for GNU Linux too (obviously). Due to the simple database structure, this application can be quickly transferred to MySQL, PostgreSQL, etc. It would probably be necessary to begin a discussion about the choice of MariaDB instead of MySQL. In this example, the choice has been made mainly because with MariaDB, I get all that I need for basic use, if PHP is an option, without installing any other software.
- HeidiSQL, as a database manager. It is written in Object Pascal and is for Windows only. In my opinion, it's the best available. I can install HeidiSQL with MariaDB (it's an option during the installation process) or the last release as independent software. I haven't tested it under Wine, so I can't say more.
- GNU Emacs, as a notebook and report builder via org-mode. An org-mode file is a text file saved with the extension 'org' instead of 'txt'.
- Uniform Server (*http://www.uniformserver.com*) as provider of the Apache Web server. This is necessary to run some PHP scripts. No installation is required - just unpack and use it. Or Wampserver (*http://www.wampserver.com/en*) as an alternative. I haven't tested other alternatives.

The database

The database structure is shown in Figure 2. The primary keys are highlighted in blue and the foreign keys in green. There are six tables. The table called *Inventory* is about the reagents. What's most important about this table is the expiry date and the hazards information. With a simple query, I can know the reagents stored in a certain place, e.g., in a fridge, and the expiry date for each of them. All the information about the instruments is stored in the table called *Instruments*. Even in this case, with a simple query, I can know, for instance, all the programmed maintenance dates. The table *Samples* is the most interesting, because it stores all the information about the samples. In this table, the most important things are: the five dates for registration, delivery, sampling, analysis start and analysis end; the customer's name and the alternative for it (in case the report needs to be addressed to a company that's not the customer), the *sample notebook* and the *analysis report* (see the next section). For a small company, this table has about 3000 to 5000 rows per year (each row is a sample). The other three tables (*Customers*, *Suppliers* and *Resources*) are useful but not very interesting because they are, more or less, like a phone book. The database credentials can be settled via the *User manager*, an option of HeidiSQL. Some easy SQL query examples are shown below:

```
# Reagents expired
SELECT id,product_name,quantity,brand,no_lot,date_exp,stg_
device FROM inventory WHERE date_exp < CURDATE() ORDER BY
product_name

# Reagents that will expire within the next 30 days
SELECT id,product_name,quantity,brand,no_lot,date_exp,stg_
device FROM inventory WHERE date_exp BETWEEN CURDATE() AND
DATE_ADD(CURDATE(), INTERVAL 30 DAY) ORDER BY product_name

# Reagents stored in a certain device
SELECT id,product_name,quantity,brand,no_lot,date_exp FROM
inventory WHERE stg_device='storage-device-name' ORDER BY
product_name

# Samples from a certain customer
SELECT id,date_rgt,type,label,status FROM samples WHERE
customer_name='customer-name' ORDER BY date_rgt
```

The database structure is too long for a printed article, but it's freely (as in freedom and in beer) available at *https://github.com/astonfe/mariadb*.

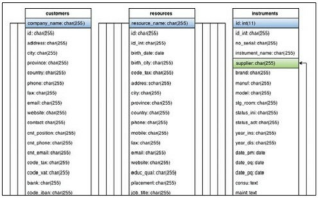

Figure 2: Database structure